

The design of analogue in-memory computing tiles

Received: 21 December 2024

Accepted: 28 October 2025

Published online: 19 December 2025



Abhairaj Singh¹✉, Manuel Le Gallo¹, Athanasios Vasilopoulos¹,
Jose Luquin², Pritish Narayanan², Geoffrey W. Burr² & Abu Sebastian¹✉

Analogue in-memory computing (AIMC) is an emerging computational approach that executes operations directly within memory arrays, reducing the need for data transfer between memory and processing units. AIMC-based accelerators are, in particular, being explored for deep neural network (DNN) inference, with the key element of such accelerators being the AIMC tile, which can be implemented using various conventional volatile charge-based and emerging non-volatile resistive memory (memristive) technologies. Here we examine the design of non-volatile memristive AIMC tiles for DNN accelerators. We explore the different components of a memristive AIMC tile and the range of mapping techniques for encoding signed multibit weights and inputs. We provide an analysis of the efficiency and accuracy of output encoding schemes, including various analogue-to-digital converter approaches. We also provide a comparative analysis of the different memory technologies being explored and projections for how technology scaling may impact key design components.

Deep neural networks (DNNs) have transformed the development of artificial intelligence (AI). Unfortunately, such approaches have substantial computational expenses, during both training and their subsequent use for inference. Indeed, the need for such extensive computational resources hindered progress in the field for nearly three decades¹. The introduction and gradual adaptation of specialized hardware such as graphics processing units has helped trigger recent AI advances, while the computational demands of training and inference with advanced AI models keep rising. For instance, training Meta's LLaMA, a massive language model with 65 billion parameters, required 2,048 NVIDIA A100 graphics processing units over 5 months, totalling approximately 2,638 MWh and emitting 1,015 tons of carbon². Deploying many instances of these same enormous models for inference, which represents over 90% of the AI hardware market, creates a considerable challenge. Furthermore, some of these models must function within systems with limited power budgets³. Widespread adoption of AI will thus remain elusive unless there is a marked improvement in the overall power consumption of AI compute substrates.

We are currently in an era of domain-specific digital accelerators for deep learning (Fig. 1a). These accelerators achieve high efficiency and computational throughput and low latency by parallelizing

operations across a large array of processing elements (PEs), using complex instructions, exploiting sparsity, adopting reduced precision and maximizing data reuse. However, even the best accelerators struggle to achieve optimal energy efficiency, often not exceeding 100 fJ op⁻¹ at high peak performance^{4,5}. The primary bottleneck is the incessant motion of synaptic weights between memory and processing units, particularly between off-chip dynamic random-access memory (DRAM) and on-chip static random-access memory (SRAM)⁶. This traffic limits energy efficiency to well below the efficiency of the actual compute operations, especially in advanced complementary metal–oxide–semiconductor (CMOS) nodes.

Several approaches are being explored to address the fundamental challenge of synaptic weight movement, including near-memory computing^{7,8} (Fig. 1b). In near-memory computing, the idea is to equip the memory units with adjacent PEs that can perform the matrix–vector multiply (MVM) operations central to DNNs, reducing the distance over which weight elements must travel. A more exploratory but potentially higher-impact approach is in-memory computing (IMC), which performs MVM operations directly within memory arrays, avoiding weight transfers. IMC inference chips consist of multiple IMC tiles⁹ (Fig. 1c). The goal of an IMC tile is to perform MVM operations on

¹IBM Research Europe, Rüschlikon, Switzerland. ²IBM Research—Almaden, San Jose, CA, USA. ✉e-mail: abhairaj.singh@ibm.com; ase@zurich.ibm.com

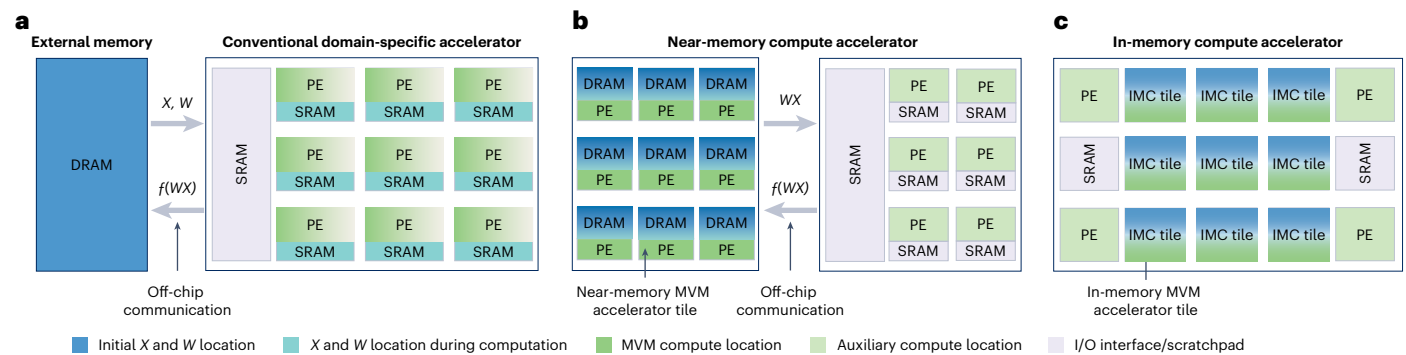


Fig. 1 | IMC-based AI accelerator. **a**, A conventional domain-specific digital accelerator for DNN inference. Both the MVM and other auxiliary computations (denoted by f) are performed in a massively parallel manner utilizing several digital PEs. Each PE has access to both its own SRAM and shared scratchpad memory, representing a significant amount of total on-chip SRAM. The processor chip also has access to an off-chip DRAM, which typically stores the synaptic weights. **b**, To alleviate the von Neumann bottleneck, there are attempts towards

near-memory computing, in which the memory unit is tightly integrated with its own PE. The MVM operations are performed in these elements, reducing the need to move weights between chips. **c**, In an IMC-based accelerator, IMC tiles perform the MVM operations within the memory arrays themselves by exploiting their physical attributes and/or array-level organization. These tiles are tightly integrated with additional PEs and on-chip SRAM. I/O, input/output.

synaptic weights, physically localized within its memory array as unit cells comprised of memory elements (Fig. 2a). Most IMC proposals have focused on SRAM memory elements, with in-memory MVM operations performed in either the digital or analogue domains. A schematic illustration of a digital IMC (DIMC) tile^{10,11} is shown in Fig. 2b. IMC that involves analogue processing is typically referred to as analogue IMC (AIMC)^{12,13} (Fig. 2c).

While AIMC synaptic weights can be realized using SRAM cells and capacitive elements, such memory devices can only encode one bit of weight data per memory element. Hence, some combination of multiple bit-lines (BLs), multiple word-lines (WLs), digital addition and/or time-multiplexing is required to implement the multiplication–accumulation between multibit activations and multibit weights. As a result, SRAM-based arrays are invariably not dense enough to hold all the weights of most DNNs on a single die under reasonable chip area constraints⁴. In this case, models with large weight capacities must be mapped across multiple SRAM-based dies, chips, nodes and racks, where high-speed interconnects become the primary challenge to ensure efficient communication between these components¹⁴. Embedded DRAM (eDRAM) can improve the on-chip weight density compared with SRAM but comes with additional challenges related to destructive DRAM read, cell leakage during inference and the associated costs of initializing and refreshing weight arrays^{15–17}.

One way to address these issues is to transition to non-volatile memory (NVM), such as resistive random-access memory (RRAM), phase-change memory (PCM) or flash memory—each of which offers higher memory density than does SRAM. Resistive memory devices such as RRAM and PCM are also referred to as memristive devices. In one implementation, by encoding multibit weights into a continuous range of analogue conductance, Ohm’s law can be used to compactly and rapidly perform the multiplication operation, followed by accumulation using Kirchhoff’s current law¹⁸. This can support both high-density and high-speed MVM operations. Moreover, NVM-based AIMC can enable power gating of an accelerator chip without needing to reload the weights.

However, these devices have limited write endurance, and programming them is costly in terms of energy and time, limiting their suitability to applications that need only occasional programming with repeated reads, such as AI inference. Consequently, NVM-based AIMC accelerators are predominantly investigated within the context of weight-stationary models that could meet the requirements of edge devices¹⁹ and large language model inference²⁰. Similar to the need to maintain DNN model accuracy despite the use of reduced precision in conventional accelerators, an important aspect of all IMC approaches

is the analysis of DNN model accuracy in the presence of analogue noise, quantization effects and other imperfections present during the MVM operations²¹.

In this Perspective, we examine the design of NVM AIMC tiles, considering the various choices and combinations available within the design space of the circuit components. We first examine the different components of a memristive AIMC tile and the range of mapping techniques for encoding signed multibit weights and inputs. We then consider the challenges and opportunities associated with designing these components. We also provide an analysis of the efficiency and accuracy of output encoding schemes, including various analogue-to-digital converter (ADC) approaches. Finally, we present a comparative analysis of the different memory technologies being explored and projections for how technology scaling may impact tile efficiency and weight capacity. (Previous review articles focusing on particular aspects of this technology are also available^{22–28}, including those with a primary focus on unit-cell configurations^{22–24} and benchmarking methodology^{27,28}).

AIMC tiles

An AIMC tile implements MVM operations where an input vector is multiplied by a weight matrix stored in a synaptic unit-cell crossbar array within the tile. For example, a column vector of inputs would produce an entire row vector of MVM outputs. Inputs, weights and outputs can all be signed, multibit or analogue quantities, and proper encoding and design are needed to accommodate all cases. Although some works on SRAM-based IMC have explored floating-point precision of inputs and weights, by handling the exponent and mantissa representations separately using a combination of time, digital and voltage (analogue) domain computing blocks^{29–31}, this requires considerably higher memory capacity and computation energy than the more conventional designs using integer precision³¹. Therefore, we focus on integer precision to illustrate the different encoding schemes.

Weight encoding

The encoding schemes for mapping signed multibit weights with B_w -bit precision onto a memory array can be generally divided into two categories: (1) analogue differential storage and (2) digital bit-slicing storage³². In both schemes, several elementary sub-unit cells are required to map each signed weight, so collectively we can group these into a single weight unit cell, where these cells are arranged in a crossbar array configuration (Fig. 3a). This crossbar configuration can potentially utilize either selector-based or selectorless sub-unit-cell implementations. Selectorless cells offer highly dense cross-point memory, but greatly

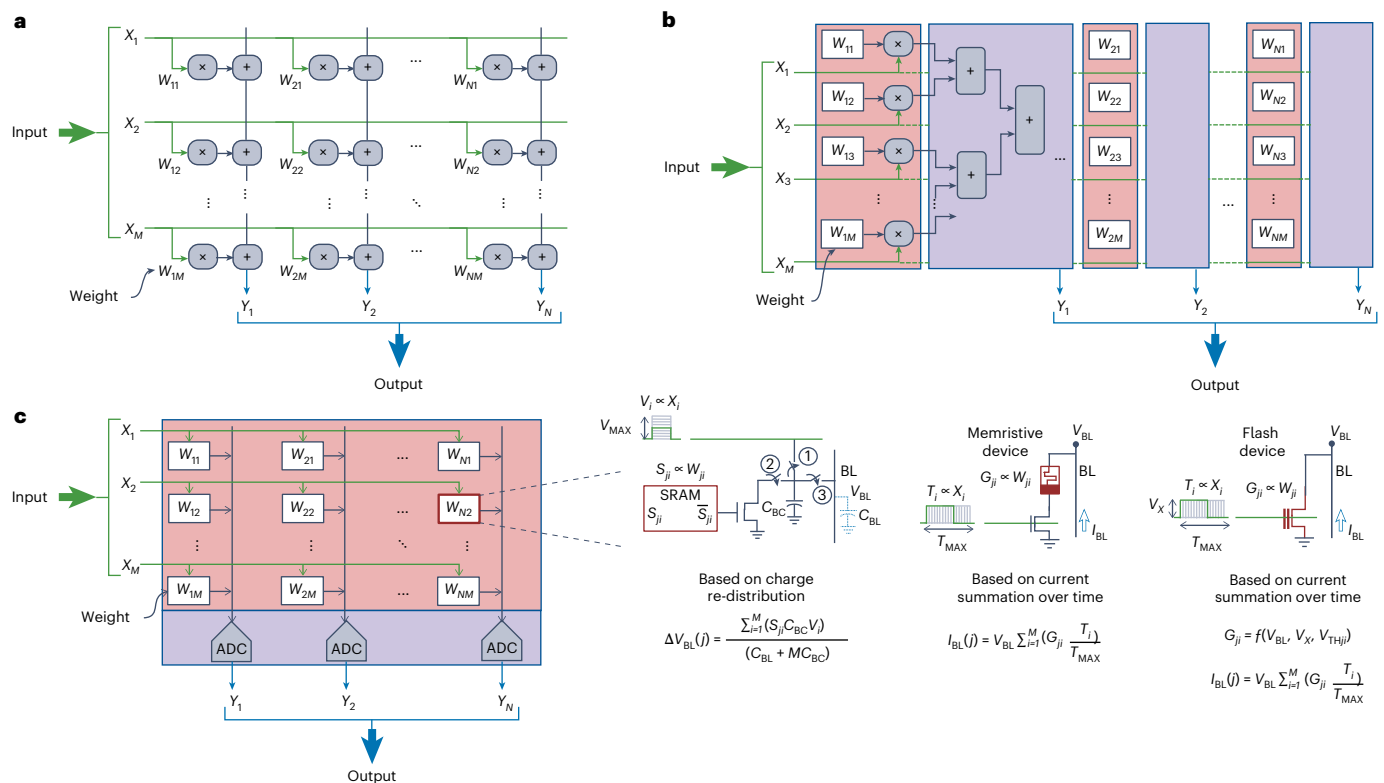


Fig. 2 | IMC tile. **a**, An IMC tile performs MVM operations using physically localized synaptic weights W_{ij} realized using unit cells of memory elements that interact with input activation signals X_i to generate output signals Y_j . **b**, In a DIMC tile, weights are encoded into binary memory elements such as SRAM cells. Specialized read mechanisms and additional digital logic circuitry perform the multiply and accumulate operations. **c**, In an AIMC tile, the accumulation takes place in the analogue domain using Kirchhoff's current law to sum current or charge which can be digitized using ADCs. While an AIMC tile can comprise SRAM-based unit cells, there are also embodiments where the weights are encoded into a continuous range of analogue conductances using memristive or flash-memory-based unit cells, allowing Ohm's law to be used to multiply

incoming activations by multibit weight values. Regarding the parameters introduced to illustrate the use of various storage elements for weight encoding, W_{ij} is mapped to S_{ij} , which represents the binary state of the SRAM bit cell, and to G_{ij} , which represents the effective readout conductance value of the memristive or flash device. C_{BC} is a capacitive element, and C_{BL} is the BL capacitance. V_{THij} denotes the threshold voltage of the flash device. X_i is mapped to V_i in a voltage-modulation encoding and to T_i (with a fixed voltage amplitude V_X) in a time-modulation encoding, while V_{MAX} and T_{MAX} denote the maximum (full-scale) input voltage and time activations, respectively. ΔV_{BL} and I_{BL} represent the partial analogue MVM results available on the BL in voltage- and current-based computing, respectively.

complicate the design of AIMC tiles. Being unable to isolate row and column activations, such a memory is highly susceptible to the adverse effects of even slightly resistive wiring, especially given the larger read currents exhibited during MVM computation. As a result, unwanted voltage gradients along the wires lead to excessive sneak paths during programming and MVM read operations. Therefore, the use of selector-based unit cells has been generally preferred in the current state-of-the-art AIMC chips^{12,31,33}.

In the analogue differential storage approach, each weight unit cell includes two sets of sub-unit cells: one for positive weights and a second for negative weights. The conductance of the device(s) within the sub-unit cell that corresponds to the polarity of the weight being encoded is programmed to be proportional to the weight magnitude, while the device(s) in the opposing set are programmed into the minimum-possible-conductance state. To reduce programming variation, multiple devices can be used per sign, with their conductances adding up. For zero weights, all devices are preferably programmed to a minimum conductance. If multiple devices are available to encode the weight magnitude, additional options for distributing the weight magnitude across these devices become available as well, allowing optimization for placement accuracy, long-term conductance stability and other considerations^{34,35}. In the digital bit-slicing weight-storage approach, weights are typically mapped using a two's complement format³⁶. Each sub-unit cell, assumed to be a single-level capacity (SLC) device, represents one bit, with logic 0 and 1 programmed to low RESET

(G_{RESET}) and high SET (G_{SET}) conductance states, respectively. For an 8-bit signed weight, eight sub-unit cells are needed, each with varying bit significance, which can be scaled using analogue or digital factors in the peripheral circuitry.

Input encoding

Inputs can be encoded in the voltage or time domain. Voltage encoding schemes offer fast sensing but are difficult to implement due to inefficient digital-to-analogue converters (DACs), device nonlinearity and IR drop considerations³⁷. We focus on the more prevalent time-based encoding schemes in this section.

Encoding schemes for mapping multibit inputs with B_X -bit precision can be grouped into three approaches on the basis of bit parallelism: (1) bit serial (BS), where each bit is represented as a separate activation signal, (2) bit parallel (BP), where all bits are combined into a single signal, and (3) bit hybrid (BH)^{13,38}, where groups of bits are represented by separate signals. The BS and BP approaches are shown in Fig. 3a. In the BS approach, logic 1 or 0 is represented by high- or low-voltage pulses. In the BP approach, a single pulse represents all bits, with its duration proportional to the input value. The pulse length is defined with respect to the smallest time-step T_{STEP} , corresponding to the least significant bit (LSB), and increases by multiples of T_{STEP} for higher bits, ranging from 0 to $2^{B_X-1} \times T_{STEP}$. The BH approach breaks the B_X -bit input into two (or more) parts, leading to multiple durations, each ranging from 0 to $2^{B_{Yj}-1} \times T_{STEP}$, where $\sum_j B_{Yj} = B_X$. While the BS

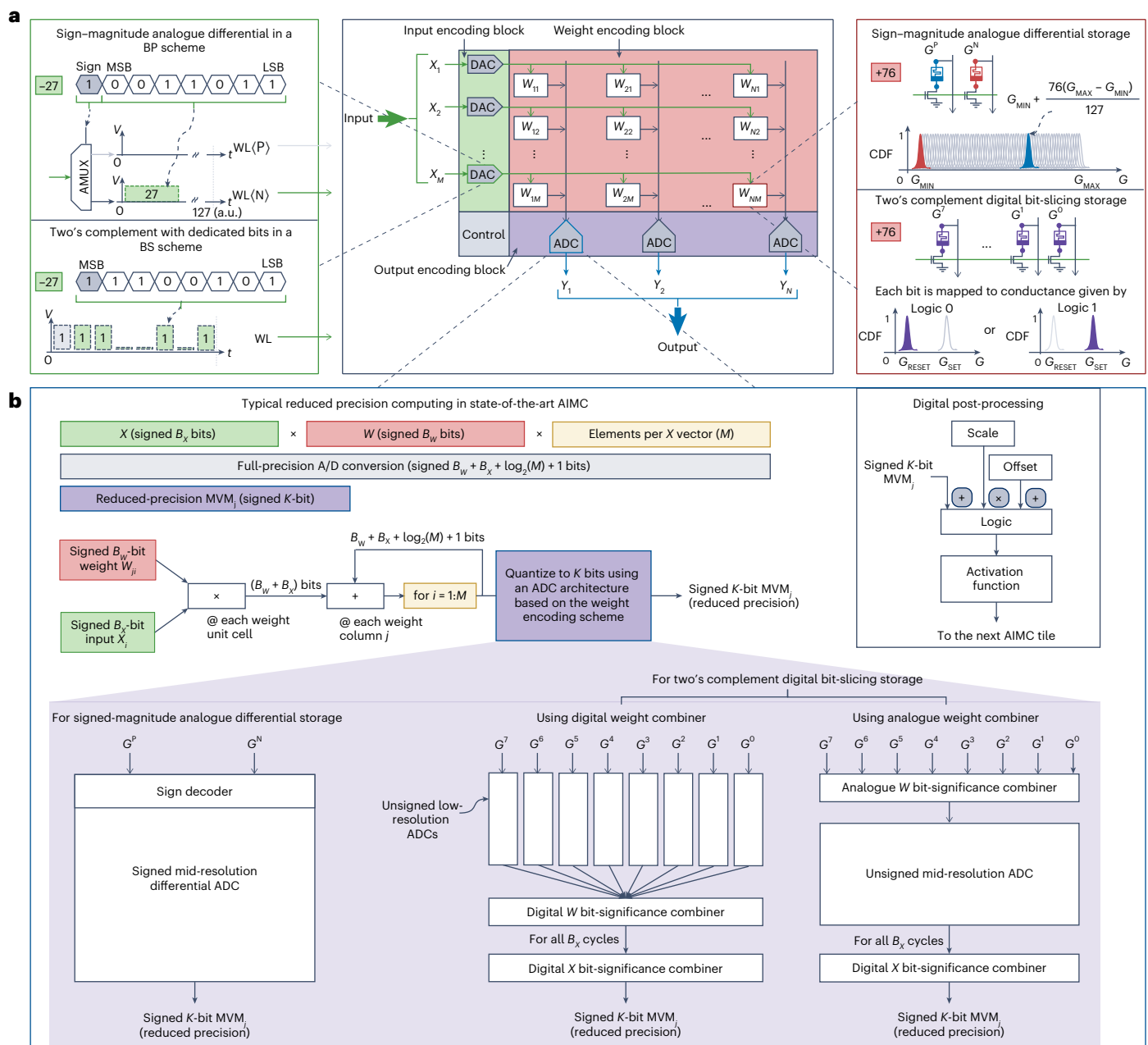


Fig. 3 | Detailed overview of an AIMC tile. a, The main components of an AIMC tile are a synaptic unit-cell crossbar array for weight encoding, an input encoding block and an output encoding block. We assume 8-bit precision of both weights and inputs to illustrate the different encoding schemes. Each weight element W_{ji} is stored in a group of sub-unit cells, collectively denoted by a weight unit cell. Weight bits can be encoded in parallel in an analogue differential manner, where dedicated sub-unit cells are used for each sign, or serially in a two's complement format onto a group of sub-unit cells. During an MVM operation, the input encoding block receives the activation inputs X_i from the previous tile

and buffers the input bits into a DAC block. The DAC encodes the input bits in the analogue domain in the sign-magnitude format by representing all input bits in parallel using time modulation (BP encoding) or in two's complement format by representing all inputs serially using discrete pulses (BS encoding) or a hybrid of the two (BH encoding). AMUX, analogue multiplexer. a.u., arbitrary units. CDF, cumulative distribution function; P, positive; N, negative. **b**, The encoded inputs interact with the weights in the memory array to generate analogue MVM outputs Y_j , which are digitized using low- or mid-resolution ADCs along with possible analogue pre-conversion and digital post-conversion steps.

scheme is advantageous versus BP in terms of total integration time, it requires additional approaches to handle the 2^i error amplification between phases. The BH approach seeks to bridge this gap and can be used to reduce the number of activation cycles and total integration time, despite requiring longer activation periods than the BS scheme.

In addition to magnitude encoding, input sign information must be accommodated to perform a four-quadrant MVM operation. Activation signals can be applied to either the gate or source of the select transistor in sub-unit cells to support these operations. Activating

the gate (WL) enables binary representation, with WL activation and deactivation representing logic 1 and 0, respectively. Activating the source (source-line, SL) enables ternary representation by setting the BL voltage to a read value (V_{READ}) and adjusting the SL using reference value (V_{REF}) to $V_{\text{REF}} + V_{\text{READ}}$, V_{REF} or $V_{\text{REF}} - V_{\text{READ}}$ to represent +1, 0 or -1, respectively. This differential input sign encoding is applicable to all three encoding schemes, but requires a symmetric read of the conductances, and entails additional complexity in the array and peripheral circuitry³⁹. The BS scheme also supports two's complement encoding

of the sign using one extra phase, but this added phase tends to be particularly sensitive to quantization and other errors. More straightforward alternatives to sign encoding include using twice the number of phases or twice the number of unit cells^{40,41}, incurring a twofold penalty in either time or area.

Output encoding

The output encoding block carries out the analogue-to-digital (A/D) conversion of the MVM output, incorporating both analogue pre-conversion and digital post-conversion steps for voltage-to-current conversion and to manage weighted partial MVM results. Optionally, it may be followed by digital post-processing units for tasks such as scaling and offset correction, applying activation functions and buffering data for the subsequent AIMC tile^{42,43}. Recent AIMC-based DNN (and numerous reduced-precision digital accelerator) implementations show that the output encoding does not need to retain the full precision; mid-resolution A/D conversion is often sufficient to achieve near-software accuracy for advanced AI applications²¹.

A/D conversion can use either voltage or current ADCs. Voltage ADCs need integration capacitors for signal storage, enabling ADC sharing and easing pitch-matching¹³. Current ADCs allow immediate conversion without large capacitors but require dedicated pitch-matched units per column⁴³. As an example of handling weighted partial MVM outputs, an AIMC tile using bit-sliced weights and BS/BH input encoding distributes bits across columns and input cycles. Bit significance can be managed via (1) analogue pre-processing, where scaled column currents or capacitors feed a mid-resolution ADC^{44,45}, or (2) digital post-processing, where low-resolution ADC outputs are combined using shift-and-add logic¹³ (Fig. 3b). Input bit significance is handled similarly; voltage sensing enables weighted accumulation across phases, while current sensing requires immediate conversion and risks amplifying quantization errors⁴⁶.

Design challenges for AIMC tiles

Device and circuit non-idealities

The primary factors contributing to accuracy degradation in AIMC tiles stem from the non-ideal behaviour of both memory devices and peripheral circuitry involved in the analogue computation of MVM operations. Imperfections in the input activation signals, as well as variability in the conductance or capacitive properties of memory devices, directly affect the correctness of the analogue outputs generated by the crossbar array. Additionally, inaccuracies introduced during the digitization of these analogue signals further compound the degradation of the final MVM result. Non-idealities in charge-based memories such as SRAMs have been extensively discussed in previous review articles, with particular emphasis on various unit-cell configurations and their accuracy and efficiency²³. Recent studies employing capacitor-based unit cells have demonstrated that such architectures exhibit strong robustness against process and device-level variations^{23,47}. On the other hand, non-idealities in memristive devices include programming errors, finite conductance ratios due to non-zero minimum conductance value (G_{MIN}), and temporal conductance drift⁴⁸. Programming errors occur when the written conductance deviates from the intended value, affecting readout accuracy and reducing the sensing margin. Techniques such as using multiple devices with equal bit significance or binary true and complementary data storage can mitigate these errors, but at the cost of reduced weight density. Iterative program-and-verify methods are often necessary to achieve target conductance levels^{35,49}.

Limited conductance ratios reduce sensing margins. While six-transistor (6T) SRAM offers a read current ratio near 1,000, those of PCM and RRAM range from 10 to 100 (refs. 50,51), and that of spin-transfer torque MRAM often falls below 2 (ref. 52). A non-zero G_{MIN} causes input-dependent offsets in MVM, requiring costly digital correction that impacts speed, energy and area. Additionally, conductance drift in PCM and RRAM introduces errors. Global drift can be

corrected via column sampling⁵³, but device-level variability remains difficult to compensate³². The use of enhanced device structures, such as projected PCM devices, can mitigate the temporal conductance drift, but at the expense of conductance ratio⁵⁴.

Circuit non-idealities include wire parasitic mismatches, CMOS process variations and design variations. Wire parasitic mismatches result from the finite resistance and capacitance of interconnect wires in the memory array, affecting propagation delays and signal amplitude. Capacitance influences the transient behaviour of analogue signals, while resistance impacts both the transient response and signal magnitude due to IR drop, leading to inconsistent activation times of select transistors and variable degradation of analogue signals across different memory locations. This effect is input dependent, with the impact of IR drop depending on the input vector conditions. Certain circuit-level mitigation techniques, such as matching the wire parasitics at the array level and reducing supply noise, can be utilized to boost signal-to-noise ratio in the presence of input-dependent variations^{52,55}. CMOS process variations can also affect the analogue readout and sensing circuits. While analogue components such as amplifiers and reference blocks can be calibrated, additional variations from the memory array may reduce sensing margins. Temperature and voltage fluctuations can alter CMOS drive strengths and wire resistance, leading to design variations. Additionally, the ADC conversion process itself is subject to error, and the digital shift-and-add process, as described in the previous section, can amplify initially small errors.

Overall, these non-idealities contribute to diminished sensing margins and introduce nonlinearity in the analogue MVM output^{13,56}. Hardware-aware training or fine-tuning of DNNs, which encapsulates some of these effects in the forward pass, can be used to make models resilient to deterministic (for example, IR drop) or random (for example, read noise) sources of noise and help preserve model accuracy^{21,57,58}.

Energy, speed and area efficiency

Energy efficiency depends on operational voltage, tile size, data conversions and wire parasitics. Higher read voltage improves dynamic range and sensing margins, while higher periphery voltage boosts transistor headroom and noise margins. However, dynamic energy rises quadratically in digital (CV^2) and linearly in analogue circuits. ADCs are major energy consumers. Larger tiles help amortize ADC costs but increase IR drop and dynamic power, and may reduce efficiency when mapping smaller DNN layers. D/A conversion consumes energy for data pre-processing and input encoding, driven by power-hungry components such as large WL/SL drivers and high-frequency clocks. Higher D/A resolution increases DAC energy use and prolongs static current duration in elements such as crossbar arrays and operational transconductance amplifiers (OTAs) in ADCs. In advanced nodes, DAC latency also prolongs leakage consumption in digital circuits such as oscillators and counters. Additional energy is spent on long wire signalling, line stabilization and capacitive output driving during MVM operations.

Latency is mainly determined by the input encoding scheme and influenced by (1) the internal clock period, (2) worst-case input transition time and (3) OTA response time on the BL. Low bit parallelism or multiphase MVM schemes require extra settling time per cycle. A/D resolution also impacts latency (time-based ADCs see exponential delays with higher resolution), but this is often masked by the input encoding latency, as conversion occurs in parallel.

Finally, the AIMC tile area is determined by the memory array and the input and output encoding blocks. Enlarging the memory array within an AIMC tile improves area efficiency and reduces inter-tile communication but introduces wire parasitic challenges. Finite RC effects can cause IR drops, delay mismatches and signal integrity issues, especially for analogue signals. Larger arrays also increase input settling times and may require buffering to preserve pulse width. Programming memristive devices typically requires high voltages (~2 V for PCM, 3–4 V for RRAM) and 100–500- μA currents, necessitating thick

oxide drivers and large select transistors⁴³, which further limits array size. The input and output encoding blocks must ideally fit within the weight unit-cell pitch to avoid the latency increases associated with column multiplexing. BS schemes are relatively simple to implement with shift registers, but BH and BP require pulse-width modulation circuits with additional circuitry. Additionally, a considerable area is required for A/D conversion and related circuits, including reference blocks and calibration circuits. Pitch-matching also applies to ADCs, posing challenges for long and thin ADC designs, where parasitic wires can further impact conversion accuracy⁵⁹.

Analogue-to-digital converters

The output encoding block can be divided into three stages: pre-conversion, conversion and post-conversion (Fig. 4a). The pre-conversion stage performs a direct voltage or current sensing and optionally translates to another analogue domain (current to voltage) and combines weighted analogue quantities using an analogue weight combiner. The conversion stage involves A/D conversion using ADCs. The post-conversion stage performs final data formatting and optionally shift-and-add operations using digital logic. In voltage sensing, an analogue voltage V_{MVM} proportional to the MVM result is directly obtained from the array, and usually stored on a capacitor⁵⁶. A current sense approach holds the access line at a fixed read voltage (typically using an OTA) while acquiring a current. This current can then be directly digitized using a current-based ADC⁴³, or integrated into a capacitor and then subsequently digitized using a voltage-based ADC⁴¹. Below, we describe different ADC architectures in these classes and present their advantages and disadvantages.

Voltage-based ADCs

The flash ADC, successive-approximation register ADC (SAR ADC), voltage-sense-amplifier-based ADC (VSA ADC) and voltage-to-time converter ADC (VTC ADC) are all examples of voltage-based ADCs (Fig. 4b). The flash ADC compares the analogue voltage with all possible reference levels in a single cycle using multiple comparators^{45,60,61}. For an n -bit ADC, $2^n - 1$ comparators are required, generating a binary vector that is decoded by a thermometer decoder to produce the final output. While the latency remains constant regardless of resolution, the area and energy grow exponentially with the number of comparators, flip-flops, interconnects, thermometer logic and size of the resistive ladder generating reference levels. Additionally, post-processing is needed to decode the thermometer code into the final ADC output. Area and energy requirements imply that mid-range resolution (~8-bit) flash ADCs are not a good fit for AIMC, but low-resolution flash ADCs have been used with additional conversion steps for more bits^{25,62}. The SAR ADC performs successive comparisons of analogue values using a binary search and an adaptive reference set based on previous decisions^{41,63–67}. As bit resolution increases, latency grows linearly with the number of comparison cycles, but area and energy consumption grow semi-exponentially due to the increasing size of the capacitive DAC, along with a linear increase in comparator, calibration and SAR logic size. Moderate resolution, low-to-medium energy consumption and low latency make SAR a good fit for AIMC. Optimizations such as bridge capacitors⁶⁸ or shifted voltage references⁶⁹ offer area saving with increasing bit-widths. The VSA ADC converts multiphase analogue outputs of varying bit significance through successive voltage comparisons with tunable resolution^{38,62,70,71}. It processes inputs in multiple cycles, generating partial analogue outputs, often combined with a BS input encoding scheme where the number of cycles is equal to B_x , or a BH scheme, reducing the number of cycles. Bit significance scaling can occur pre-conversion in the analogue domain (via weighted current mirrors or capacitors) or post-conversion in the digital domain (using shift-and-add logic). The number of conversions and bit resolution are tunable, thus balancing accuracy and efficiency. VTC ADCs have an additional pre-conversion stage that converts the acquired

voltage into a time signal, which can then be quantized using a clock, a free-running oscillator or a look-up table. In some scenarios involving simple activation functions such as ReLU, the generated time pulse can be clipped and consumed downstream as the pulse-width input for the next NN layer, avoiding the intermediate analogue-to-digital and digital-to-time steps entirely^{33,72}. Time-based ADCs suffer from exponential time increases with bit resolution, but this can be alleviated using fast counting mechanisms at moderate resolution of interest, thanks to technology scaling.

An inherent advantage of all voltage-based architectures over current-based ones is that storing the output as a voltage allows some amount of ADC sharing across different access lines if needed, mitigating tight pitch-matching constraints. For instance, the reference capacitor bank, comparator and SAR logic block of the SAR ADC can be shared across multiple columns, which are converted sequentially. Latency increases, but can be carefully managed through pipelining if individual A/D conversions can be considerably faster than the overall MVM integration time. Sharing approaches for the flash ADC are similarly straightforward. For the VSA ADC, this approach is possible, but depends on the exact scheme being used and the internals of the architecture.

Current-based ADCs

Current-based ADCs directly convert an output current I_{MVM} on the access lines proportional to the MVM result into a digital value. The integrate-and-fire circuit ADC (IFC ADC), current-controlled oscillator ADC (CCO ADC) and current-sense-amplifier-based ADC (CSA ADC) follow this approach (Fig. 4c).

The IFC ADC integrates current over a capacitor to generate a voltage, firing a pulse when this voltage crosses a threshold and resetting the capacitor^{73–75}. The number of pulses is proportional to the current magnitude. Higher bit resolution is achieved by increasing pulse counts, resulting in exponential growth in latency and energy consumption, while the area remains nearly constant. While the IFC leaks continuously and is subject to the threshold voltage variation of the discharge transistor, a variant known as the charge-injection cell uses a comparator and feedback mechanism to discharge the capacitor only when a pulse is fired⁷⁶. The CCO ADC generates oscillations in an oscillator with a frequency proportional to the input current⁴³. The number of oscillations produced in a fixed time determines the digital output. The use of a dedicated oscillator and the high frequency at which it operates allow the CCO to achieve a higher digital resolution over similar time-scales compared with IFC approaches, but it does require nonlinear compensation at high input currents. The CCO design is also more compact and energy efficient than the SAR or flash ADCs since it does not require either capacitance banks or multiple comparators, with most of the power consumed by the OTA and digital counter. As bit resolution increases, the area remains nearly constant (except for a larger counter), but latency and energy grow exponentially due to the rise in maximum oscillation counts. Like the VSA ADC, the CSA ADC converts multiphase analogue outputs of varying bit significance through successive current comparisons with tunable resolution^{44,77,78}. The column current is either mirrored or directly fed into the CSA, while a reference block provides tunable reference signals. The CSA size and number of reference blocks grow exponentially with increasing resolution. Most energy is consumed by the OTA and CSA, often exceeding that of the VSA ADC¹³. Bit significance scaling can occur pre-conversion in the analogue domain via weighted current mirrors (though this poses additional mismatch and calibration challenges)⁴⁴ or post-conversion digitally (using shift-and-add logic)^{13,37}, balancing accuracy and efficiency.

Direct current-to-digital conversion precludes circuit sharing and potentially multiplies positional input significance errors in BS and BH schemes due to residual quantization error effects. In most cases, handling the sign of the MVM requires separate components (except for the case of the charge-injection cell that handles differential currents)

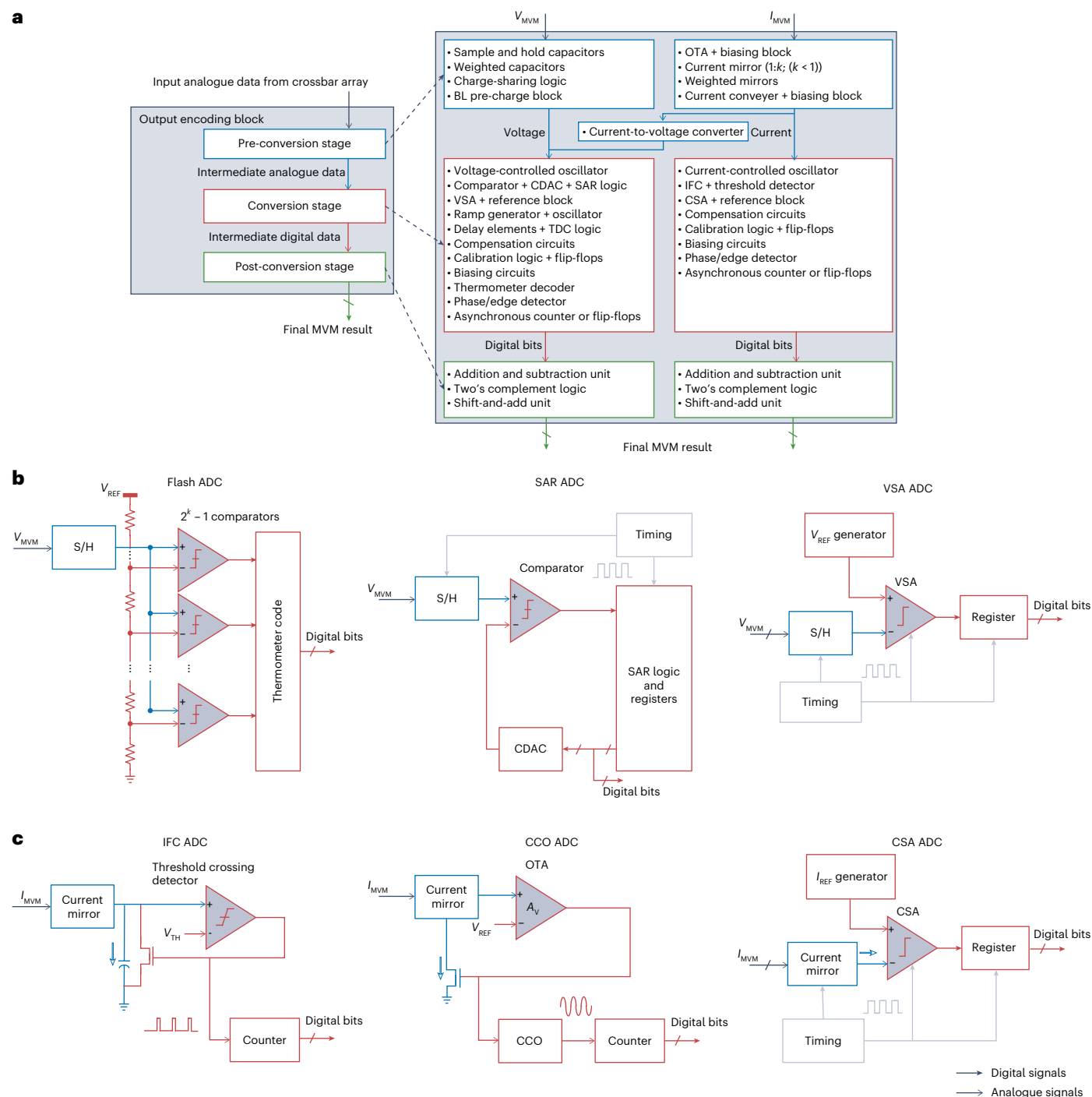


Fig. 4 | Output encoding block and ADC types. a, The output encoding block can be broadly divided into three stages depending on the underlying tasks performed: a pre-conversion stage, a conversion stage and a post-conversion stage. On the right are lists of the relevant circuit blocks used within each output

encoding stage. CDAC, capacitive DAC. TDC, time-to-digital converter. **b**, Flash ADC, SAR ADC and VSA ADC, the most popular voltage-based ADCs. **c**, IFC ADC, CCO ADC and CSA ADC, the most popular current-based ADCs. S/H, sample and hold capacitors.

whose outputs need to be combined in post-processing with additional mismatch correction. Yet the area benefits, lower energy cost and scalability make current-based architectures worthy of consideration in certain scenarios.

ADC design considerations

Unlike high-speed applications, AIMC-specific ADCs prioritize compact, energy-efficient designs. In dense PCM-based 256×256 arrays, ADCs and support circuits can consume over 40% of area and 50–60% of

energy⁴³, limiting weight density. ADCs must deliver mid-range resolution with minimal overhead and match the narrow column pitch, often with aspect ratios of >25 , complicating layout and signal routing. For BP encoding, slow oscillator-based ADCs are preferred for efficiency, while comparator-based ADCs offer better latency scaling but with higher area and power costs. In BS encoding, small-pitch, low-cost, unsigned low-resolution comparator-based ADCs per sub-cell are favoured for speed and efficiency, though at the expense of higher quantization errors from repeated conversions and shift-and-add operations.

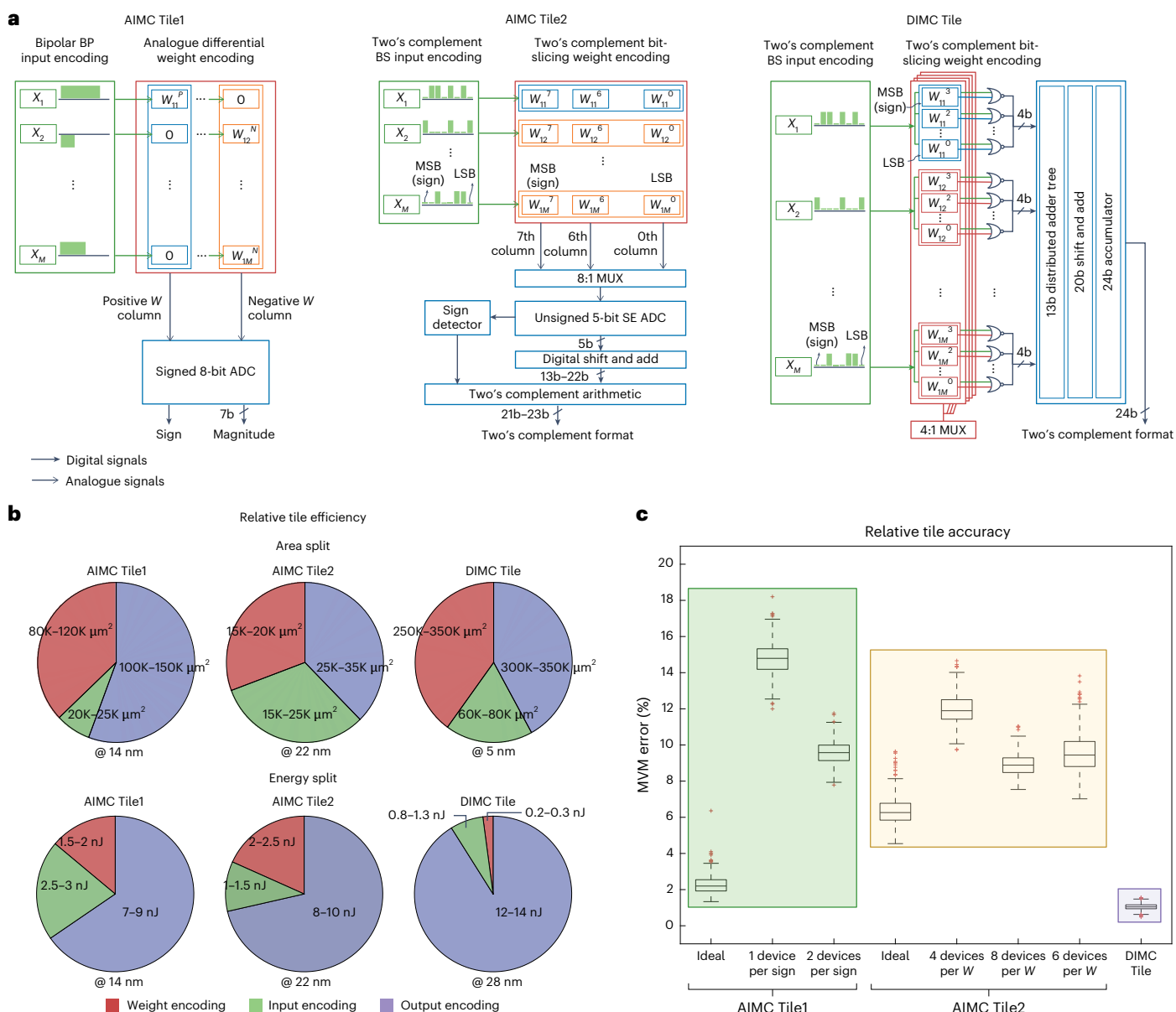


Fig. 5 | Comparative study of prominent IMC tiles. **a**, Schematic illustration of the two most prominent AIMC tile architectures. AIMC Tile1 is based on a purely analogue-based encoding approach using sign–magnitude data format where a bipolar BP input encoding scheme is coupled with an analogue differential weight encoding scheme while utilizing a signed differential mid-resolution oscillator-based ADC (signed ADC) for each weight column. AIMC Tile2 is based on a more discrete approach using two's complement data format where a unipolar BS input encoding scheme is coupled with a column-wise bit-slicing weight encoding scheme while utilizing a dedicated unsigned single-ended low-resolution comparator-based ADC (SE ADC) for each weight-bit column (several such ADCs for each weight column or one such ADC shared among different array columns within one weight column), followed by digital shift and add, and a two's complement arithmetic unit. Also shown is DIMC Tile, based on a fully digital approach using the two's complement data format where a unipolar BS input encoding scheme is coupled with a row-wise bit-slicing weight encoding scheme while utilizing a distributed adder tree, shift and add, and an accumulator

for each weight column. **b**, A comparative study of the component-wise area (top) and energy split (bottom) of the three tile architectures. The area and energy numbers are estimated on the basis of data from the reference works, extrapolated to a 256×256 tile configuration and 8-bit input and weight precision^{26,43,44,81,82}. For the DIMC tile, we use $256 \times 256 \times 4 \times 4$ 1-bit SRAM cells for area calculations (see Supplementary Note 1 for details). K indicates 1,000. **c**, A comparative study of the MVM error expected using these tile architectures. 'Ideal' refers to having ideal 8-bit weights with input/output quantization according to **a**. '1/2 devices per sign' refers to having 1/2 devices cumulatively dedicated to a positive or negative signed weight. '4 devices per W' refers to having four devices, each storing two weight bits. '6 devices per W' refers to four devices storing one bit each (starting from the most significant bit (MSB)), and the remaining four LSB bits stored in two devices, each storing two bits. '8 devices per W' refers to having eight devices, each storing one bit. Each box-plot is generated with 500 repetitions of the simulation model described in Supplementary Note 2. Centre line, median; box limits, upper and lower quartiles; whiskers, $1.5 \times$ interquartile range; points, outliers.

The ADC must handle non-idealities in the analogue MVM signal, including small margins, wide dynamic range and uncertain polarity (for example, in ternary SL activation). To ensure linearity, unit-cell voltages are fixed during reads using OTAs or current conveyors, though large RC loading and dynamic ranges can cause long

settling times or require high static power. Signal margins are further reduced by programming errors, IR drop and device drift. Techniques such as high-resistance weight encoding, adaptive reference drift compensation^{33,79} and program–verify using ADCs^{12,33} help mitigate these issues. Program–verify also demands high ADC resolution (for

example, 12 bit for 256 4-bit weights) and dynamic scaling, adding area overhead. However, infrequent tuning and long read times can ease resolution needs, and gradient-descent-based tuning⁸⁰ can aid in correcting individual cell errors efficiently.

Input encoding schemes pose additional challenges for the OTA. BS and BH encoding require fast, multiple settling cycles with varying inputs, increasing design cost and quantization errors. BP encoding needs a high OTA bandwidth for rapid settling, enlarging the size and power consumption. Additionally, weight slicing demands precise analogue MVM scaling, which is costly to calibrate^{38,45}, while post-conversion digitization can further amplify quantization errors³⁷.

IMC tile efficiency and weight capacity

We begin by analysing how the memory array, input and output encoding blocks affect efficiency and accuracy. We then review recent trends in efficiency, weight capacity and unit-cell size in IMC-based DNN accelerators, and conclude with projections for how technology scaling may impact key design components.

Relative contribution of IMC tile sub-blocks

The computational efficiency and accuracy of IMC tiles are strongly influenced by the encoding schemes for inputs and weights, which are also tied to the output processing methods. To explore these relationships, three different tile designs are analysed (Fig. 5a). AIMC Tile1 uses an analogue-leaning approach where multibit inputs and weights are represented as analogue quantities. AIMC Tile2 employs a digital-leaning approach with multibit inputs and weights represented as discrete quantities. Both AIMC tiles are based on memristive devices. DIMC Tile is a fully digital SRAM-based IMC tile performing bitwise multiplication of inputs and weights. The ADC design, topology and architecture in AIMC tiles depend on the encoding schemes. Each tile assumes signed 8-bit precision for inputs and weights, with a 256×256 weight matrix. The designs draw from recent literature^{26,43,44,81,82} and we analyse component-wise contributions to the overall area and energy (see Supplementary Note 1 for details).

The top half of Fig. 5b shows relative component-wise area contributions. Despite variations in technology nodes and design constraints, a few key observations emerge. Output encoding comprising ADC, pre- and post-conversion blocks in AIMC and adder trees/accumulators in DIMC dominates area in all cases. Analogue memory/SRAM blocks follow, then input encoding. Though memristive devices can enable denser storage than can SRAM, the DIMC design uses a more advanced node; still, its weight encoding takes up a larger area share than in AIMC. Among AIMC tiles, Tile2 has a higher input encoding area due to more complex WL drivers, two's complement logic and possibly input bit reordering.

Regarding energy consumption, the bottom half of Fig. 5b shows that the output encoding block is the dominant energy consumer, accounting for 60–70% of energy in AIMC tiles and over 80% in the DIMC tile. In AIMC Tile1, static power from the OTA and dynamic power from the oscillator and counter circuits contribute significantly. AIMC Tile2, despite using low-resolution comparator-based ADCs with low static power, requires multiple A/D conversions per MVM operation, leading to higher relative energy consumption than Tile1. In the DIMC

tile, the output block's high energy use stems from substantial digital transitions and leakage power due to low-threshold-voltage CMOS devices⁸¹. The memory array contributes significantly less energy, at 15–20% in AIMC tiles and minimal in the DIMC tile. For AIMC tiles, this energy depends on memristive device resistance and wire parasitics during pre-charging, while DIMC tiles achieve efficient and fast operations due to low RC loading and minimal active rows. The absolute energy consumption of input encoding blocks also differs between AIMC tiles. In Tile1, using the BP scheme, energy consumption is typically lower due to a single small ΔV_{READ} above or below V_{REF} , resulting in minimal switching energy. In contrast, Tile2, which employs the BS scheme, requires multiple large full-scale voltage switches, leading to higher CV^2 switching energy. This increased switching activity can reach up to 50% due to the two's complement representation of input bits.

Although analogue weight storage in the Tile1 configuration is not feasible using SRAM unit cells, these cells can be utilized for digital bit-sliced weight storage in Tile2. The encoding schemes may remain similar; however, the MVM read operation can be based on voltage^{45,64,83–85}, time⁸⁶ or even charge^{47,66,87–89}, as in the case of capacitor-based unit cells. Employing SRAM unit cells in Tile2 would result in an order-of-magnitude higher area overhead for the memory array compared with memristive devices ('Projections for efficiency with technology scaling'), while theoretically maintaining similar contributions from the input and output encoding circuits.

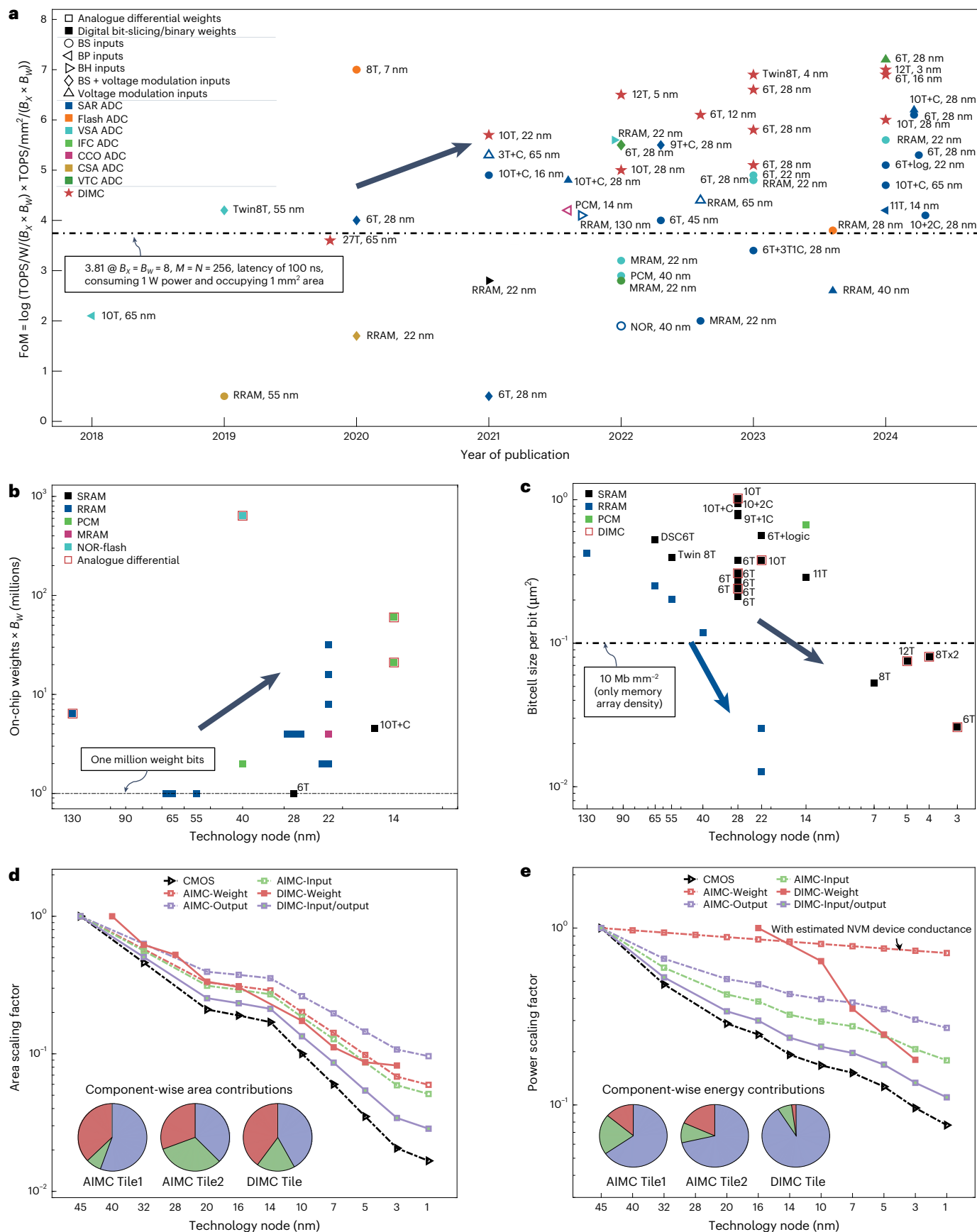
In terms of MVM accuracy, Fig. 5c illustrates significant variation between AIMC and DIMC tiles. Here, the relative MVM error is the deviation of the computed MVM result compared with the error obtained using floating-point precision, where an MVM error of zero corresponds to employing floating-point input and weight precision with an ideal A/D conversion. In AIMC Tile1, the average MVM error increases from 2% under ideal conditions to 15% under non-ideal conditions, primarily due to programming errors and IR drop. Introducing two devices per sign to store weights¹² reduces the error to below 10%, though at the expense of weight density and capacity. AIMC Tile2 has an inherently higher MVM error (6%) under ideal conditions due to quantization from digital-type encoding and global scaling factors. Non-idealities further increase its error, similar to Tile1, but this can be mitigated by increasing the number of devices per weight. In contrast, DIMC tiles achieve a low MVM error of 1%, benefiting from digital encoding of inputs and weights and the use of a full-precision adder tree and shift-and-add block. (See Supplementary Notes 2 and 3 for further details of the noise models and a component-wise analysis.)

Projections for efficiency with technology scaling

Over the past decade, numerous IMC-based DNN accelerators have emerged, leveraging diverse memory technologies, unit-cell designs and encoding schemes on advanced CMOS nodes. Since 2018, the area and energy efficiency of AIMC and DIMC tiles have improved exponentially, driven by innovations in device and circuit design and in scaling (Fig. 6a). Most AIMC designs use mid-/low-resolution comparator-based ADCs (for example, SAR, flash, VSA, CSA) with bit-slicing, voltage modulation or hybrid input encoding, while oscillator-based ADCs and BP encoding remain less common. DIMC solutions show a strong figure of merit (FoM), leveraging technology and voltage scaling for improved

Fig. 6 | Recent trends and technological projections. **a**, Recently published IMC tiles along with a normalized FoM that combines their reported area and energy efficiency. Also highlighted are the associated weight and input encoding schemes, ADC design, memory device, sub-unit-cell configuration and technology node. For SRAM-based tiles sub-unit cells are indicated by the number of transistors, while for other technologies they are identified by the type of NVM used. A linear trend in the FoM on a logarithmic scale signifies exponential improvement. **b**, Plot capturing the increase in on-chip weight capacity, denoted by the number of weight bits, with technology scaling. Memristive and flash memories offer at least an order-of-magnitude-higher weight capacity compared

with SRAM-based IMC. **c**, An exponential reduction in area per unit weight bit owing to circuit innovations and technology scaling. Within AIMC, RRAMs offer superior weight density owing to their compact structure and multibit capability. **d, e**, Recent trends and projections for area (**d**) and power (**e**) scaling in relation to technology scaling. CMOS transistor scaling is shown as a reference, representing the lower bound for any other scaling factor. A higher proportion of analogue components diminishes the benefits of technology scaling, as reflected in the scaling of input and output encoding blocks compared with DIMC. To easily visualize the impact on the overall efficiency metrics, we also show the relative component-wise area and energy contributions (the same as presented in Fig. 5).



efficiency. At advanced nodes, DIMC tiles are notably more energy and area efficient than AIMC tiles due to low MVM latency and sub-threshold operation. Recent work reports up to tenfold gains in efficiency, though this comes with limited on-chip weight capacity (<100 KB), requiring frequent weight reloading for larger models (>1 MB). The overhead of weight transfers, despite excellent macrolevel FoMs, may become a critical bottleneck and warrants thorough architectural evaluation.

There are also promising improvements for the on-chip weight capacity, particularly for AIMC. Figure 6b shows recently proposed IMC accelerators that can support more than one million on-chip weight bits (none of the DIMC works could be included due to their lower weight capacity) and Fig. 6c shows the weight density in terms of unit-cell area per bit. On the basis of the same CMOS technology, a more than tenfold improvement is observed between memristive units and SRAM. This is because a single transistor-based memristive unit cell can store one or even more bits, whereas a multitransistor SRAM cell can store only one bit. In fact, a multilevel capacity (MLC)-based RRAM cell on 22-nm CMOS technology¹³ is nearly the same size (estimated from the unit cell occupying 53F² area) as a 6T SRAM cell based on 3-nm CMOS technology⁹⁰, implying nearly twice the weight density.

Projections based on reported IMC tile efficiency and memristive device resistance suggest distinct trends for DIMC- and AIMC-based accelerators with technology scaling (see Supplementary Note 4 for more details). For DIMC, further improvements are anticipated due to advancements in voltage scaling, CMOS logic and SRAM scaling beyond 3-nm technology (Fig. 6d,e). These improvements are estimated to provide up to 1.4-fold gains in density and 1.2-fold gains in energy efficiency with each CMOS technology node^{91,92}. However, estimating the full potential of DIMC-based accelerators is challenging due to limited data on fully integrated systems, particularly regarding on-chip data communication. Technology scaling in AIMC can improve device resistance and offer area and power gains of up to 1.2-fold and 1.1-fold per CMOS node beyond 22 nm. AIMC also benefits from a nearly ten times smaller weight encoding area than DIMC, thanks to memristive devices, an advantage expected to grow with multibit encoding. However, scaling introduces challenges such as increased wire RC, longer OTA response times, higher static energy, IR drop and reduced noise margins, all of which can degrade MVM accuracy.

Finally, to support large DNN models, on-chip weight capacity can be increased using three-dimensional (3D) memory-based tiles. Recent IMC demonstrations have explored 3D NAND flash^{93,94} and NOR flash⁹⁵, which support both SLC and MLC and are compatible with digital bit-slicing and analogue differential weight encoding in AIMC tiles. NAND-flash-based 3D structures are particularly promising, with an expected annual areal density increase of 1.48-fold due to further scaling and stacking⁹⁴. While 3D memristive devices are under active development^{96,97}, 3D SRAM remains limited by manufacturing challenges⁹⁸.

Outlook

Recent silicon-verified demonstrations of AIMC tiles have exhibited exponential advancements in precision, energy efficiency, area efficiency, on-chip weight capacity and weight density, driven by circuit innovations and advancements in CMOS technology scaling. While the wire parasitics increase substantially with technology scaling, high-resistance devices are becoming more favourable for weight encoding. The resulting lower column currents help mitigate accuracy degradation due to IR drops and concurrently reduce overall energy consumption.

For future implementations, digital-intensive data conversion topologies, particularly in ADC designs, are highly desirable when implemented with advanced CMOS technology nodes and can further leverage the advantages of technology scaling. Notably, compact time-based ADCs such as ramp-based VTCs and controlled oscillator-based designs offer inherent advantages over traditional voltage or current comparator-based ADCs. This can be attributed to

the exponential improvement in time resolution enabled by technology scaling, while operating voltage and threshold levels remain relatively stagnant, resulting in limited scaling benefits for reference and comparator circuits. Design techniques such as component sharing, replacing resource-intensive digital on-chip calibration with compact analogue calibration schemes⁹⁹ and reducing the number of A/D conversions through the use of analogue combiner circuits are becoming increasingly preferred to realize compact and energy-efficient ADC designs.

In the near term, AIMC based on switched-capacitor SRAM is likely to remain a focus of research due to its resilience to device-level variations, robustness against IR drop-induced accuracy degradation, and mature manufacturing infrastructure. Hybrid designs combining SRAM and memristive devices may also offer a compelling trade-off between robustness, density and energy efficiency, paving the way for practical and scalable AIMC solutions^{31,100}.

However, we have shown that, even with relatively mature CMOS nodes, highly dense memristive and 3D flash memory technologies offer an order-of-magnitude improvement in weight density and on-chip storage capacity compared with SRAM implemented using the most advanced CMOS technology. This, along with the support of novel circuit- and algorithm-level signal-to-noise ratio enhancement techniques, suggests that long-term future advancements in memristive and flash memories using advanced CMOS technology nodes could enable the support of large models fully on-chip, paving the way for more efficient DNN accelerators²⁰.

Data availability

The data that support the plots within this paper are available from the corresponding authors upon reasonable request.

References

- Hooker, S. The hardware lottery. *Commun. ACM* **64**, 58–65 (2021).
- Touvron, H. et al. LLaMA: open and efficient foundation language models. Preprint at <https://arxiv.org/abs/2302.13971> (2023).
- Chen, J. & Ran, X. Deep learning with edge computing: a review. *Proc. IEEE* **107**, 1655–1674 (2019).
- Murmann, B. Mixed-signal computing for deep neural network inference. *IEEE Trans. Very Large Scale Integr. VLSI Syst.* **29**, 3–13 (2021).
- Reuther, A. et al. AI and ML accelerator survey and trends. In *2022 IEEE High Performance Extreme Computing Conference (HPEC)* 1–10 (IEEE, 2022).
- Horowitz, M. Computing's energy problem (and what we can do about it). In *International Solid-State Circuits Conference (ISSCC)* 10–14 (IEEE, 2014).
- He, M. et al. Newton: a DRAM-maker's accelerator-in-memory (AiM) architecture for machine learning. In *IEEE/ACM International Symposium on Microarchitecture (MICRO)* 372–385 (IEEE, 2020).
- Kwon, D. et al. A 1nm 1.25V 8Gb, 16Gb/s/pin GDDR6-based accelerator-in-memory supporting 1TFLOPS MAC operation and various activation functions for deep-learning applications. *IEEE J. Solid State Circuits* **58**, 291–302 (2022).
- Sebastian, A., Le Gallo, M., Khaddam-Aljameh, R. & Eleftheriou, E. Memory devices and applications for in-memory computing. *Nat. Nanotechnol.* **15**, 529–544 (2020).
- d-Matrix Corsair Redefines Performance and Efficiency for AI Inference at Scale White Paper* (d-Matrix, 2025); <https://d-matrix.ai/pdf/d-Matrix-WhitePaper-Technical-FINAL.pdf>
- Hager, P. A. et al. 11.3 Metis AIPU: a 12nm 15TOPS/W 209.6 TOPS SoC for cost-and energy-efficient inference at the edge. In *2024 IEEE International Solid-State Circuits Conference (ISSCC)* Vol. 67, 212–214 (IEEE, 2024).
- Le Gallo, M. et al. A 64-core mixed-signal in-memory compute chip based on phase-change memory for deep neural network inference. *Nat. Electron.* **6**, 680–693 (2023).

13. Hsu, H.-H. et al. A nonvolatile AI-edge processor with SLC–MLC hybrid ReRAM compute-in-memory macro using current–voltage-hybrid readout scheme. *IEEE J. Solid State Circuits* **59**, 116–127 (2023).
14. Abts, D. et al. The Groq software-defined scale-out tensor streaming multiprocessor: from chips-to-systems architectural overview. In *Hot Chips Symposium (HCS)* 1–69 (IEEE Computer Society, 2022).
15. Xie, S. et al. eDRAM-CIM: compute-in-memory design with reconfigurable embedded-dynamic-memory array realizing adaptive data converters and charge-domain computing. In *International Solid-State Circuits Conference (ISSCC)* Vol. 64, 248–250 (IEEE, 2021).
16. Kim, S. et al. Scaling-CIM: an eDRAM-based in-memory-computing accelerator with dynamic-scaling ADC for SQNR-boosting and layer-wise adaptive bit-truncation. In *2023 IEEE Symposium on VLSI Technology and Circuits (VLSI Technology and Circuits)* 1–2 (IEEE, 2023).
17. Song, J. et al. A calibration-free 15-level/cell eDRAM computing-in-memory macro with 3T1C current-programmed dynamic-cascoded MLC achieving 233-to-304-TOPS/W 4b MAC. In *Custom Integrated Circuits Conference (CICC)* 1–2 (IEEE, 2023).
18. Burr, G. W., Sebastian, A., Ando, T. & Haensch, W. Ohm's law + Kirchhoff's current law = better AI: neural-network processing done in memory with analog circuits will save energy. *IEEE Spectrum* **58**, 44–49 (2021).
19. Boybat-Kara, I. et al. Heterogeneous embedded neural processing units utilizing PCM-based analog in-memory computing. In *IEEE International Electron Devices Meeting* 1–4 (IEEE, 2024).
20. Büchel, J. et al. Efficient scaling of large language models with mixture of experts and 3D analog in-memory computing. *Nat. Comput. Sci.* **5**, 13–26 (2025).
21. Rasch, M. J. et al. Hardware-aware training for large-scale and diverse deep learning inference workloads using in-memory computing-based accelerators. *Nat. Commun.* **14**, 5282 (2023).
22. Mittal, S., Verma, G., Kaushik, B. & Khanday, F. A. A survey of SRAM-based in-memory computing techniques and applications. *J. Syst. Archit.* **119**, 102276 (2021).
23. Jhang, C.-J., Xue, C.-X., Hung, J.-M., Chang, F.-C. & Chang, M.-F. Challenges and trends of SRAM-based computing-in-memory for AI edge devices. *IEEE Trans. Circuits Syst. I* **68**, 1773–1786 (2021).
24. Spetalnick, S. & Raychowdhury, A. A practical design-space analysis of compute-in-memory with SRAM. *IEEE Trans. Circuits Syst. I* **69**, 1466–1479 (2022).
25. Yu, S., Sun, X., Peng, X. & Huang, S. Compute-in-memory with emerging nonvolatile-memories: challenges and prospects. In *Custom Integrated Circuits Conference (CICC)* 1–4 (IEEE, 2020).
26. Hung, J.-M., Jhang, C.-J., Wu, P.-C., Chiu, Y.-C. & Chang, M.-F. Challenges and trends of nonvolatile in-memory-computation circuits for AI edge devices. *IEEE Open J. Solid State Circuits Soc.* **1**, 171–183 (2021).
27. Shanbhag, N. R. & Roy, S. K. Benchmarking in-memory computing architectures. *IEEE Open J. Solid State Circuits Soc.* **2**, 288–300 (2022).
28. Sun, J., Houshmand, P. & Verhelst, M. Analog or digital in-memory computing? Benchmarking through quantitative modeling. In *International Conference on Computer Aided Design (ICCAD)* 1–9 (IEEE, 2023).
29. Wu, P.-C. et al. A 22nm 832Kb hybrid-domain floating-point SRAM in-memory-compute macro with 16.2–70.2 TFLOPS/W for high-accuracy AI-edge devices. In *International Solid-State Circuits Conference (ISSCC)* 126–128 (IEEE, 2023).
30. Guo, A. et al. A 28 nm 128-kb exponent- and mantissa-computation-in-memory dual-macro for floating-point and INT CNNs. *IEEE J. Solid State Circuits* **60**, 3639–3654 (2025).
31. Khwa, W.-S. et al. A mixed-precision memristor and SRAM compute-in-memory AI processor. *Nature* **639**, 617–623 (2025).
32. Le Gallo, M. et al. Precision of bit slicing with in-memory computing based on analog phase-change memory crossbars. *Neuromorphic Comput. Eng.* **2**, 014009 (2022).
33. Ambrogio, S. et al. An analog-AI chip for energy-efficient speech recognition and transcription. *Nature* **620**, 768–775 (2023).
34. Mackin, C. et al. Optimised weight programming for analogue memory-based deep neural networks. *Nat. Commun.* **13**, 3765 (2022).
35. Vasilopoulos, A. et al. Exploiting the state dependency of conductance variations in memristive devices for accurate in-memory computing. *IEEE Trans. Electron Devices* **70**, 6279–6285 (2023).
36. Xiao, T. P., Bennett, C. H., Feinberg, B., Agarwal, S. & Marinella, M. J. Analog architectures for neural network acceleration based on non-volatile memory. *Appl. Phys. Rev.* **7**, 031301 (2020).
37. Lee, J., Valavi, H., Tang, Y. & Verma, N. Fully row/column-parallel in-memory computing SRAM macro employing capacitor-based mixed-signal computation with 5-b inputs. In *Symposium on VLSI Circuits* 1–2 (IEEE, 2021).
38. Xue, C.-X. et al. A 22nm 4Mb 8b-precision ReRAM computing-in-memory macro with 11.91 to 195.7TOPS/W for tiny AI edge devices. In *International Solid-State Circuits Conference (ISSCC)* Vol. 64, 245–247 (IEEE, 2021).
39. Le Gallo, M. et al. Demonstration of 4-quadrant analog in-memory matrix multiplication in a single modulation. *npj Unconv. Comput.* **1**, 11 (2024).
40. Jung, S. et al. A crossbar array of magnetoresistive memory devices for in-memory computing. *Nature* **601**, 211–216 (2022).
41. Fick, L., Skrzyniarz, S., Parikh, M., Henry, M. B. & Fick, D. Analog matrix processor for edge AI real-time video analytics. In *International Solid-State Circuits Conference (ISSCC)* Vol. 65, 260–262 (IEEE, 2022).
42. Li, W., Sun, X., Jiang, H., Huang, S. & Yu, S. A 40nm RRAM compute-in-memory macro featuring on-chip write-verify and offset-cancelling ADC references. In *European Solid State Circuits Conference (ESSCIRC)* 79–82 (IEEE, 2021).
43. Khaddam-Aljameh, R. et al. HERMES-core—a 1.59-TOPS/mm² PCM on 14-nm CMOS in-memory compute core using 300-ps/LSB linearized CCO-based ADCs. *J. Solid State Circuits* **57**, 1027–1038 (2022).
44. Xue, C.-X. et al. A 22nm 2Mb ReRAM compute-in-memory macro with 121–28TOPS/W for multibit MAC computing for tiny AI edge devices. In *International Solid-State Circuits Conference (ISSCC)* 244–246 (IEEE, 2020).
45. Sinangil, M. E. et al. A 7-nm compute-in-memory SRAM macro supporting multi-bit input, weight and output and achieving 351TOPS/W and 372.4 GOPS. *J. Solid State Circuits* **56**, 188–198 (2020).
46. Wan, W. et al. A compute-in-memory chip based on resistive random-access memory. *Nature* **608**, 504–512 (2022).
47. Jia, H., Valavi, H., Tang, Y., Zhang, J. & Verma, N. A programmable heterogeneous microprocessor based on bit-scalable in-memory computing. *IEEE J. Solid State Circuits* **55**, 2609–2621 (2020).
48. Burr, G. W. et al. Recent progress in phase-change memory technology. *IEEE J. Emerg. Sel. Top. Circuits Syst.* **6**, 146–162 (2016).
49. Sun, X. et al. PCM-based analog compute-in-memory: impact of device non-idealities on inference accuracy. *IEEE Trans. Electron Devices* **68**, 5585–5591 (2021).
50. Le Gallo, M. & Sebastian, A. An overview of phase-change memory device physics. *J. Phys. D* **53**, 213002 (2020).
51. Xia, Q. & Yang, J. J. Memristive crossbar arrays for brain-inspired computing. *Nat. Mater.* **18**, 309–323 (2019).

52. Deaville, P., Zhang, B. & Verma, N. A fully row/column-parallel in-memory computing macro in foundry MRAM with differential readout for noise rejection. *IEEE J. Solid State Circuits* **59**, 2070–2080 (2024).
53. Le Gallo, M., Sebastian, A., Cherubini, G., Giefers, H. & Eleftheriou, E. Compressed sensing with approximate message passing using in-memory computing. *IEEE Trans. Electron Devices* **65**, 4304–4312 (2018).
54. Giannopoulos, I. et al. 8-bit precision in-memory multiplication with projected phase-change memory. In *International Electron Devices Meeting (IEDM) 27.7* (IEEE, 2018).
55. Roy, S. K. et al. Compute SNDR-boosted 22-nm MRAM-based in-memory computing macro using statistical error compensation. *IEEE J. Solid State Circuits* **60**, 1092–1102 (2025).
56. Jia, H. et al. Scalable and programmable neural network inference accelerator based on in-memory computing. *IEEE J. Solid State Circuits* **57**, 198–211 (2021).
57. Miyashita, D., Kousai, S., Suzuki, T. & Deguchi, J. A neuromorphic chip optimized for deep learning and CMOS technology with time-domain analog and digital mixed-signal processing. *IEEE J. Solid State Circuits* **52**, 2679–2689 (2017).
58. Joshi, V. et al. Accurate deep neural network inference using computational phase-change memory. *Nat. Commun.* **11**, 2473 (2020).
59. Vignali, R. et al. Designing circuits for AiMC based on non-volatile memories: a tutorial brief on trade-off and strategies for ADCs and DACs co-design. *IEEE Trans. Circuits Syst. II* **71**, 1650–1655 (2023).
60. Yin, S. et al. High-throughput in-memory computing for binary deep neural networks with monolithically integrated RRAM and 90-nm CMOS. *IEEE Trans. Electron Devices* **67**, 4185–4192 (2020).
61. Yoon, J.-H. et al. A 40-nm, 64-Kb, 56.67TOPS/W voltage-sensing computing-in-memory/digital RRAM macro supporting iterative write with verification and online read-disturb detection. *IEEE J. Solid State Circuits* **57**, 68–79 (2021).
62. Khwa, W.-S. et al. A 40-nm, 2M-cell, 8b-precision, hybrid SLC-MLC PCM computing-in-memory macro with 20.5–65.0TOPS/W for tiny-AI edge devices. In *International Solid-State Circuits Conference (ISSCC)* Vol. 65, 1–3 (IEEE, 2022).
63. Liu, Q. et al. A fully integrated analog ReRAM based 78.4TOPS/W compute-in-memory chip with fully parallel MAC computing. In *International Solid-State Circuits Conference (ISSCC)* 500–502 (IEEE, 2020).
64. Si, X. et al. A 28nm 64Kb 6T SRAM computing-in-memory macro with 8b MAC operation for AI edge chips. In *International Solid-State Circuits Conference (ISSCC)* 246–248 (IEEE, 2020).
65. Caselli, M., Bhattacharjee, D., Mallik, A., Debacker, P. & Verkest, D. Tiny ci-SAR A/D converter for deep neural networks in analog in-memory computation. In *International Symposium on Circuits and Systems (ISCAS)* 1823–1827 (IEEE, 2022).
66. Chen, Z. et al. CAP-RAM: a charge-domain in-memory computing 6T-SRAM for accurate and precision-programmable CNN inference. *IEEE J. Solid State Circuits* **56**, 1924–1935 (2021).
67. Biswas, A., Sanghvi, H., Mehendale, M. & Preet, G. An area-efficient 6T-SRAM based compute-in-memory architecture with reconfigurable SAR ADCs for energy-efficient deep neural networks in edge ML applications. In *Custom Integrated Circuits Conference (CICC)* 1–2 (IEEE, 2022).
68. Um, J.-Y., Kim, Y.-J., Song, E.-W., Sim, J.-Y. & Park, H.-J. A digital-domain calibration of split-capacitor DAC for a differential SAR ADC without additional analog circuits. *IEEE Trans. Circuits Syst. I* **60**, 2845–2856 (2013).
69. Kull, L. et al. A 3.1mW 8b 1.2GS/s single-channel asynchronous SAR ADC with alternate comparators for enhanced speed in 32nm digital SOI CMOS. *IEEE J. Solid State Circuits* **48**, 3049–3058 (2013).
70. Chiu, Y.-C. et al. A 22nm 4Mb STT-MRAM data-encrypted near-memory computation macro with a 192GB/s read-and-decryption bandwidth and 25.1–55.1TOPS/W 8b MAC for AI operations. In *International Solid-State Circuits Conference (ISSCC)* Vol. 65, 178–180 (IEEE, 2022).
71. Hung, J.-M. et al. A four-megabit compute-in-memory macro with eight-bit precision based on CMOS and resistive random-access memory for AI edge devices. *Nat. Electron.* **4**, 921–930 (2021).
72. Narayanan, P. et al. Fully on-chip MAC at 14 nm enabled by accurate row-wise programming of PCM-based weights and parallel vector-transport in duration-format. *IEEE Trans. Electron Devices* **68**, 6629–6636 (2021).
73. Liu, C. et al. A spiking neuromorphic design with resistive crossbar. In *Design Automation Conference (DAC)* 1–6 (IEEE, 2015).
74. Liu, C. et al. A memristor crossbar based computing engine optimized for high speed and accuracy. In *IEEE Computer Society Annual Symposium on VLSI (ISVLSI)* 110–115 (IEEE, 2016).
75. Yan, B. et al. RRAM-based spiking nonvolatile computing-in-memory processing engine with precision-configurable in situ nonlinear activation. In *Symposium on VLSI Technology* T86–T87 (IEEE, 2019).
76. Elshazly, A., Rao, S., Young, B. & Hanumolu, P. K. A noise-shaping time-to-digital converter using switched-ring oscillators—analysis, design, and measurement techniques. *IEEE J. Solid State Circuits* **49**, 1184–1197 (2014).
77. Chen, W.-H. et al. A 65nm 1Mb nonvolatile computing-in-memory ReRAM macro with sub-16ns multiply-and-accumulate for binary DNN AI edge processors. In *International Solid-State Circuits Conference (ISSCC)* 494–496 (IEEE, 2018).
78. Chen, W.-H. et al. CMOS-integrated memristive non-volatile computing-in-memory for AI edge processors. *Nat. Electron.* **2**, 420–428 (2019).
79. Antolini, A. et al. Controlled acceleration of PCM cells time drift through on-chip current-induced annealing for AIMC multilevel MVM computation. *IEEE Trans. Electron Devices* **72**, 215–221 (2025).
80. Büchel, J. et al. Programming weights to analog in-memory computing cores by direct minimization of the matrix-vector multiplication error. *IEEE J. Emerg. Sel. Top. Circuits Syst.* **13**, 1052–1061 (2023).
81. Wang, D. et al. DIMC: 2219TOPS/W 2569F2/b digital in-memory computing macro in 28nm based on approximate arithmetic hardware. In *International Solid-State Circuits Conference (ISSCC)* Vol. 65, 266–268 (IEEE, 2022).
82. Fujiwara, H. et al. A 5-nm 254-TOPS/W 221-TOPS/mm² fully-digital computing-in-memory macro supporting wide-range dynamic-voltage-frequency scaling and simultaneous MAC and write operations. In *International Solid-State Circuits Conference (ISSCC)* Vol. 65, 1–3 (IEEE, 2022).
83. Biswas, A. & Chandrakasan, A. P. CONV-SRAM: an energy-efficient SRAM with in-memory dot-product computation for low-power convolutional neural networks. *IEEE J. Solid State Circuits* **54**, 217–230 (2018).
84. Khwa, W.-S. et al. A 65nm 4Kb algorithm-dependent computing-in-memory SRAM unit-macro with 2.3ns and 55.8TOPS/W fully parallel product-sum operation for binary DNN edge processors. In *2018 IEEE International Solid-State Circuits Conference (ISSCC)* 496–498 (IEEE, 2018).
85. Si, X. et al. A twin-8T SRAM computation-in-memory macro for multiple-bit CNN-based machine learning. In *International Solid-State Circuits Conference (ISSCC)* 396–398 (IEEE, 2019).
86. Wu, P.-C. et al. A 28nm 1Mb time-domain computing-in-memory 6T-SRAM macro with a 6.6ns latency, 1241GOPS and 37.01TOPS/W for 8b-MAC operations for edge-AI devices. In *International Solid-State Circuits Conference (ISSCC)* Vol. 65, 1–3 (IEEE, 2022).

87. Khaddam-Aljameh, R., Francese, P.-A., Benini, L. & Eleftheriou, E. An SRAM-based multibit in-memory matrix-vector multiplier with a precision that scales linearly in area, time, and power. *IEEE Trans. Very Large Scale Integr. VLSI Syst.* **29**, 372–385 (2020).
88. Lee, C.-F. et al. A 12nm 121-TOPS/W 41.6-TOPS/mm² all digital full precision SRAM-based compute-in-memory with configurable bit-width for AI edge applications. In *Symposium on VLSI Technology and Circuits* 24–25 (IEEE, 2022).
89. Wang, H. et al. A charge domain SRAM compute-in-memory macro with C-2C ladder-based 8-bit MAC unit in 22-nm FinFET process for edge inference. *IEEE J. Solid State Circuits* **58**, 1037–1050 (2023).
90. Fujiwara, H. et al. A 3nm, 32.5 TOPS/W, 55.0 TOPS/mm² and 3.78 Mb/mm² fully-digital compute-in-memory macro supporting INT12×INT12 with a parallel-MAC architecture and foundry 6T-SRAM bit cell. In *IEEE International Solid-State Circuits Conference (ISSCC)* Vol. 67, 572–574 (IEEE, 2024).
91. Wong, H. On the CMOS device downsizing, more Moore, more than Moore, and more-than-Moore for more Moore. In *International Conference on Microelectronics (MIEL)* 9–15 (IEEE, 2021).
92. Das, U. K. & Bhattacharyya, T. K. Opportunities in device scaling for 3-nm node and beyond: FinFET versus GAA-FET versus UFET. *IEEE Trans. Electron Devices* **67**, 2633–2638 (2020).
93. Bavandpour, M., Sahay, S., Mahmoodi, M. R. & Strukov, D. B. 3D-aCortex: an ultra-compact energy-efficient neurocomputing platform based on commercial 3D-NAND flash memories. *Neuromorphic Comput. Eng.* **1**, 014001 (2021).
94. Hu, H.-W. et al. A 512Gb in-memory-computing 3D-NAND flash supporting similar-vector-matching operations on edge-AI devices. In *International Solid-State Circuits Conference (ISSCC)* Vol. 65, 138–140 (IEEE, 2022).
95. Wei, M.-L. et al. Analog computing in memory (CIM) technique for general matrix multiplication (GEMM) to support deep neural network (DNN) and cosine similarity search computing using 3D AND-type NOR flash devices. In *International Electron Devices Meeting (IEDM)* 33.3.1–33.3.4 (IEEE, 2022).
96. Du, Y. et al. Monolithic 3D integration of FeFET, hybrid CMOS logic and analog RRAM array for energy-efficient reconfigurable computing-in-memory architecture. In *Symposium on VLSI Technology and Circuits* 1–2 (IEEE, 2023).
97. Carletti, F. et al. Low-energy, high-accuracy convolutional network inference in 3D crosspoint (3DXP) arrays. In *European Solid-State Electronics Research Conference (ESSERC)* 412–415 (IEEE, 2024).
98. Yu, S. et al. Compute-in-memory: from device innovation to 3D system integration. In *European Solid-State Device Research Conference (ESSDERC)* 21–28 (IEEE, 2021).
99. Agrawal, A. et al. A switched-capacitor integer compute unit with decoupled storage and arithmetic for Cloud AI inference in 5nm CMOS. In *Symposium on VLSI Technology and Circuits* 1–2 (IEEE, 2023).
100. Wen, T.-H. et al. Fusion of memristor and digital compute-in-memory processing for energy-efficient edge computing. *Science* **384**, 325–332 (2024).

Acknowledgements

We thank P. A. Francese, S. Tsai, V. Narayanan and J. Burns for technical discussions and management support.

Author contributions

A. Singh, M.L.G. and A. Sebastian conceived of the overall research idea and coordinated the project. The manuscript was initially drafted by A. Singh, with substantial input through comments, revisions and structural guidance from A.V., M.L.G. and A. Sebastian. J.L., P.N. and G.W.B. provided critical reviews focused on the technical content and paper structure. P.N. contributed specifically to refining and streamlining the technical content, while G.W.B. offered detailed feedback on the overall presentation of the key contributions, especially in the Introduction and figures. All authors reviewed the final paper and contributed to its revision and approval.

Competing interests

The authors declare no competing interests.

Additional information

Supplementary information The online version contains supplementary material available at <https://doi.org/10.1038/s41928-025-01537-5>.

Correspondence should be addressed to Abhairaj Singh or Abu Sebastian.

Peer review information *Nature Electronics* thanks Hongyang Jia and the other, anonymous, reviewer(s) for their contribution to the peer review of this work.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

© Springer Nature Limited 2025